GPU Native Scan — Where We Landed

Branch: `feature/gpu-native-scan-task` (replaces DuckDB's CPU `duckdb_scan_task` with a Sirius-native scan + GPU decode for `.duckdb` files).

Headline

TPC-H SF=100 on GH200, warm, all 22 queries:

	GPU	CPU	GPU/CPU
Total	**5.33s**	7.08s	**1.33× (GPU wins)**
Queries where GPU wins	**15 / 22**		

Biggest per-query wins on SF=100 warm:

Q	GPU	CPU	Ratio
Q13	0.332s	0.859s	**2.59×**
Q22	0.061s	0.148s	**2.43×**
Q18	0.332s	0.682s	**2.05×**
Q14	0.101s	0.194s	**1.92×**
Q20	0.121s	0.219s	**1.81×**
Q2	0.061s	0.108s	**1.77×**
Q9	0.574s	0.902s	**1.57×**

TPC-H SF=10 on RTX 6000 (PCIe), warm, 22 queries:

Version	Total	vs baseline
old GPU pin path	~10.5s	3.7× slower than CPU
this branch	**5.37s**	**−49% vs old GPU**
DuckDB CPU	2.84s	GPU still 1.9× slower

ClickBench 10M on RTX 6000, warm, 29 queries:

Version	Total
old GPU path	3.37s
this branch	**2.10s** (−38%)

Version	Total
DuckDB CPU	1.46s

ClickBench 100-shard on GH200, warm, 16 queries tested:

	GPU	CPU
Total	5.00s	2.54s

CPU still wins ClickBench (wide-table single-scan workload — not our target), but the **previous OOM blocker is gone** (was 29.3s / OOM, now 5.0s, −83%).

What we shipped

- `gpu_native_scan_task` — bypasses DuckDB's table function for block-backed `.duckdb` storage
- GPU decode kernels: bitpacking (int8/16/32/64), dictionary, FSST, RLE, constant, uncompressed
- Fused GPU-side bitpacking metadata parse (−26% TPC-H SF10 warm alone)
- Batched two-pass string decode (eliminates per-segment sync + kernel launch)
- Dict/RLE/null-count kernel opts (−18% ClickBench)
- Direct `mmap` of `.duckdb` bypassing `BufferManager::Pin()` — **pure Sirius**, atomic fast path, zero mutex contention across parallel scan tasks
- Parallel scan tasks (N concurrent to saturate scan thread pool)

GPU kernel time at SF=100 warm Q1: 608ms — our decode is **11%** of that. Remaining 89% is cuDF hash_group_by / CUB internals. We are no longer the bottleneck on compute.

3 open questions

1. Scaling: RTX6000 → GH200, SF=10 → SF=100 — does it compound?

Yes. Direct data from the runs above:

	RTX6000 SF10 warm	GH200 SF100 warm
GPU / CPU	**0.53×** (GPU loses)	**1.33×** (GPU wins)
15/22 queries →	(GPU wins on 2)	(GPU wins on 15)

The CPU loses its L3-cache advantage as data grows; HBM bandwidth stays flat. GH200's unified-memory ATS also removes the PCIe staging cost entirely, so the scan path gets near its theoretical max. Net: **the cross-over is around SF=100**

on GH200 today. Next question is SF=1000. Does anyone have SF=1000 generated?

2. Bigger-than-memory (PCIe degrades horribly — can't test locally)

On GH200 we're confident: ATS lazy-faults the `.duckdb` mmap, pages stream in at NVLink-C2C speeds. The branch already auto-detects unified memory and skips the prefault step on GH200 (commit `27b049e`), and the 2026-04-16 GH200 run confirmed no warm regression with lazy faults. Cold was **6.9× faster** on GH200 with lazy vs prefault (2.4s vs 16.4s avg/query for TPC-H SF100).

What we don't have: an empirical SF=1000 GH200 run with cache flushed. The theory says we stream at line rate. Anyone with cycles to confirm?

On PCIe (RTX6000): unpinned mmap transfers at 6.4 GB/s (already the warm bottleneck at SF=10). Bigger-than-memory is not our problem here — it's the hardware's. We don't plan to optimize for that case.

3. With this speedup, scan/query split: do we need a GPU cache, or push on execution?

Measured split on Q1 SF=10 warm (254ms total):

	ms	%
Scan wall (6 tasks, 2 threads, ~36ms decode/task)	109	**43%**
Downstream (join / agg / sort)	145	**57%**

The prior 33/66 estimate was slightly too scan-pessimistic — real split is closer to **43/57** on Q1, and **further toward execution on Q5/Q7/Q9/Q21** (join-heavy). Aggregated across SF=10 warm, **execution dominates**.

Remaining scan optimization (ring buffer + pinned + dual-stream overlap) is worth ~58ms on Q1 SF10 — nice but not huge. **GPU cache of decoded cudf columns** would only win if we see repeated queries against the same table in a session (warm pin is already 1.5ms after DuckDB's buffer pool warms up).

Recommendation: execution first (join / agg / sort downstream), cache later. At SF=100 on GH200, our kernel is 11% of GPU time — cuDF internals are 89%. That's where the next 2× lives. Pushback welcome.

Raw CSVs + nsys profiles: `docs/super-sirius/benchmarks/2026-04-16_gh200/` on `feature/gpu-scan-duckdb-api`. Report: `gh200_sf100_clickbench_20260416.md`.